

RADIANT Workflow & Method System - Gemini Consultation Request

Date: January 31, 2026 **Version:** 5.52.58 **Purpose:** Request Gemini’s expert review on workflow/method architecture for competitive positioning through 2030

Request for Gemini

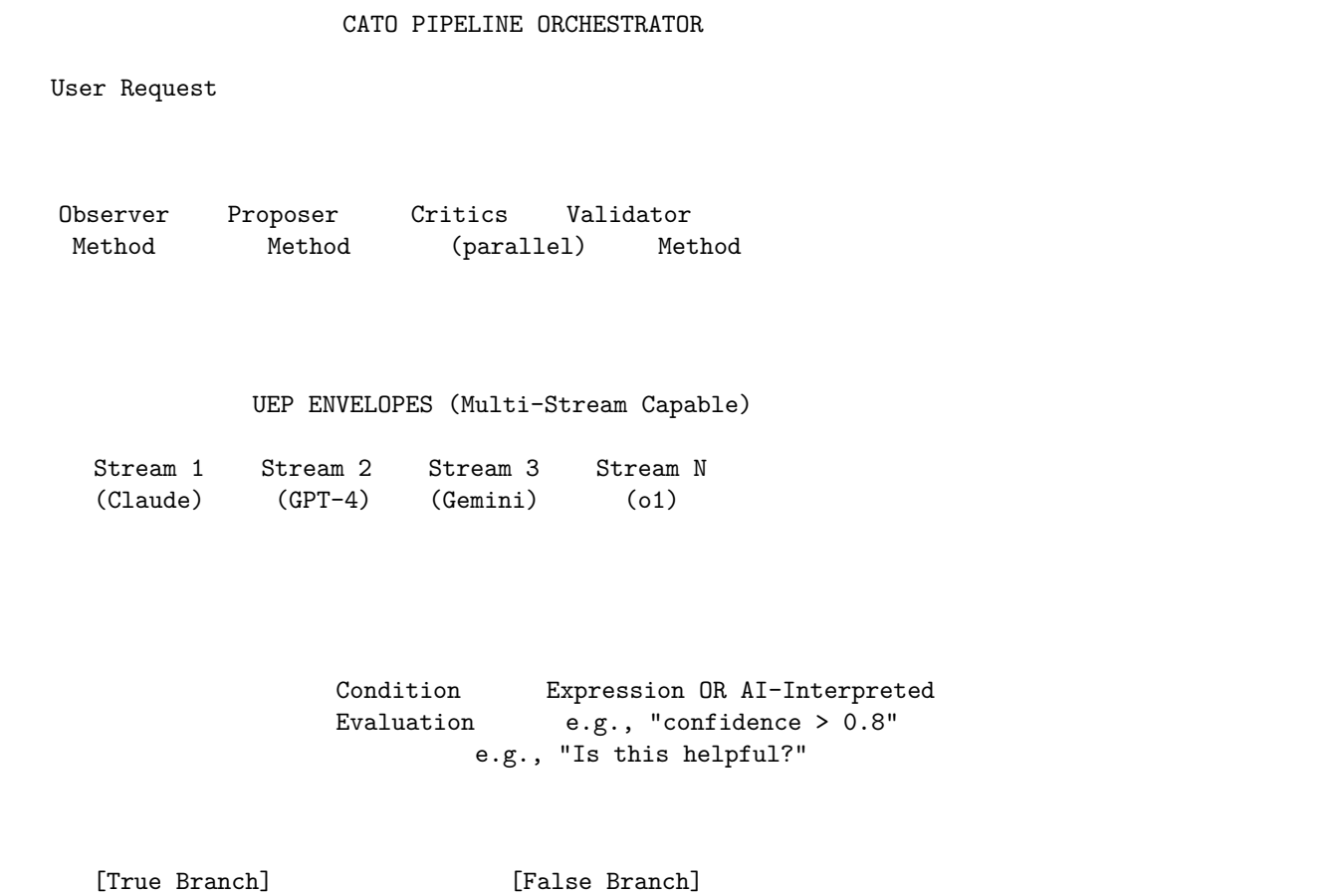
We are seeking your expert opinion on our workflow and method orchestration system. Please analyze this implementation against competitors and provide guidance on how we can get ahead now, looking forward to 2030.

Specific questions: 1. How should conditional node links be handled in general? 2. What is the optimal approach for one or more multimedia streams? 3. How does our implementation compare to competitors? 4. What capabilities should we prioritize for 2030 readiness?

System Overview

RADIANT implements a **composable AI orchestration system** with: - **70+ system workflows** with pre-configured method chains - **25+ orchestration methods** based on peer-reviewed research - **Multi-AI stream support** through UEP (Universal Envelope Protocol) - **Model-agnostic conditional routing** based on output content, not model identity - **User/tenant customizable workflows** saved for future reuse

Architecture: How Workflows and Methods Are Wired



Conditional Node Links

Condition Types

```
type ConditionType = 'expression' | 'ai_interpreted' | 'composite';

interface NodeCondition {
  conditionId: string;
  name: string;
  type: ConditionType;

  // Expression conditions (JavaScript-like evaluation)
  expression?: string; // e.g., "output.confidence > 0.8 && output.category === 'technical'"

  // AI-interpreted conditions
  aiPrompt?: string; // e.g., "Is this response helpful and on-topic?"
  aiModel?: string; // Optional: specific model for interpretation
  aiThreshold?: number; // Confidence threshold (default: 0.7)

  // Composite conditions
  operator?: 'AND' | 'OR' | 'NOT' | 'XOR';
  subConditions?: NodeCondition[];

  // Stream evaluation mode
  streamMode: StreamEvaluationMode;

  // Actions
  onTrue?: ConditionAction;
  onFalse?: ConditionAction;
}
```

Stream Evaluation Modes

When multiple AI models run in parallel, conditions must decide how to evaluate across streams:

Mode	Description	Use Case
all	ALL streams must pass	High-stakes consensus
any	ANY stream passing is sufficient	Exploratory queries
majority	>50% of streams must pass	Democratic voting
quorum	Configurable threshold (n streams)	Custom consensus
best	Select highest-scoring stream	Quality optimization
unanimous	100% agreement required	Critical decisions
weighted	Weight by confidence scores	Confidence-weighted

Key Design Decision: Model-Agnostic Conditions

Conditions evaluate OUTPUT CONTENT, not model identity.

```
// Expression evaluation context - NOTE: Only content is available
const evalContext = {
  output: output.content,
  content: output.content,
```

```

// Helper functions
hasField: (field: string) => ...,
getField: (field: string, defaultValue?) => ...,
length: () => ...,
contains: (text: string) => ...,
};
// Model identity is NOT exposed to condition evaluation

```

This means users can swap models freely without breaking workflow logic.

Multi-AI Selection

Per-Method AI Selection

Each method can specify which AI model(s) to use:

```

interface MethodExecutionConfig {
  // Single model
  modelId?: string;

  // Multiple models (parallel execution)
  modelIds?: string[];

  // Model selection strategy
  modelSelection?: 'specified' | 'router' | 'all_available' | 'tier_cascade';

  // Governance preset affects model routing
  governancePreset: 'COWBOY' | 'BALANCED' | 'PARANOID';
}

```

Workflow-Level AI Selection

Users or Cato can override AI selection for the entire workflow:

```

interface WorkflowOverrides {
  // Force specific model for all methods
  forceModelId?: string;

  // Force model tier
  forceTier?: 'fast' | 'balanced' | 'powerful' | 'reasoning';

  // Model allowlist/blocklist
  allowedModels?: string[];
  blockedModels?: string[];

  // Budget constraint
  maxCostCents?: number;
}

```

UEP Multi-AI Input/Output Streams

The Universal Envelope Protocol (UEP) v2.0 supports multi-AI streams that may be synthesized before output:

Stream Output Structure

```
interface StreamOutput {
  streamId: string;
  content: unknown;
  confidence?: number;
  latencyMs: number;
  tokensUsed: number;
  modelId?: string; // Captured but NOT used for conditions
  mode?: string;
}

interface UEPPayload {
  input: { type: 'text' | 'structured' | 'stream'; content: unknown };
  output: {
    type: 'text' | 'structured' | 'stream';
    content: unknown;
    streams?: StreamOutput[]; // Multiple parallel outputs
  };
}
```

Synthesis in Methods

Any method can synthesize multiple streams before output:

```
// MoA Synthesis Service - Mixture of Agents
class MoASynthesisService {
  async synthesize(input, params) {
    // Layer 1: Get diverse responses from fast models
    // Layer 2: Synthesize with powerful models
    // Final: Merge into single output
  }
}

// Debate Service - Multi-agent deliberation
class DebateService {
  async sparseDebate(input, params) {
    // Multiple agents debate across rounds
    // Final synthesis combines best arguments
  }
}
```

Multimedia Support

Current Status: The architecture supports multimedia, but most methods do not yet implement it.

```
interface MultimediaStream {
  streamId: string;
  mediaType: 'text' | 'image' | 'audio' | 'video' | 'document';
  content: unknown;
  mimeType?: string;
  encoding?: 'base64' | 'url' | 'raw';
  metadata?: {
    width?: number;
    height?: number;
    duration?: number;
    pages?: number;
  };
}
```

```
};
}
```

Question for Gemini: How should we handle one or more multimedia streams? What are the best practices for:
 1. Cross-modal condition evaluation (e.g., “Does this image match the text description?”) 2. Synthesis across modalities (e.g., combining text + image outputs) 3. Stream selection when mixing media types

Complete Method List

Core Pipeline Methods (Cato)

Method ID	Purpose	Implementation
method:observer:v1	Analyze user request, extract intent	CatoObserverMethod
method:proposer:v1	Generate action proposals with tool calls	CatoProposerMethod
method:critic:security:v1	Security vulnerability analysis	CatoSecurityCriticMethod
method:validator:v1	Risk assessment, triage decision	CatoValidatorMethod
method:executor:v1	Execute approved actions	CatoExecutorMethod
method:decider:v1	Final decision synthesis	CatoDeciderMethod

Orchestration Methods (70+ Total)

Service	Method	Research Reference	Configurable Parameters
semantic-entropy-service	computeEntropy	Nature 2024 - Semantic Uncertainty in LLMs	sample_count, temperature, clustering_method
se-probes-service	estimateEntropy	ICML 2024 - Semantic Entropy Probes	probe_layers, threshold, fast_mode
kernel-entropy-service	computeKDE	NeurIPS 2024 - Kernel Language Entropy	kernel, bandwidth, sample_count
self-consistency-service	multiSampleVote	Wang et al. 2022 - Self-Consistency	sample_count, temperature
poll-judge-service	evaluateWithPanel	Panel of LLMs Evaluation Framework	num_judges, scoring_criteria, aggregation
selfcheck-service	checkConsistency	SelfCheckGPT - Zero-Resource Hallucination	sample_count, consistency_threshold
routeLLM-service	routeQuery	LMSYS RouteLLM	cost_threshold, quality_floor
frugal-cascade-service	cascadeRoute	FrugalGPT 2023	model_cascade, confidence_threshold, max_escalations
debate-service	sparseDebate	Sparse Communication Topology Debate	topology, debate_rounds, temperature
multi-hallucination-service	detectHallucination	Multi-Method Hallucination Detection 2025	methods, flag_threshold

Service	Method	Research Reference	Configurable Parameters
moa-synthesis-service	synthesize	Together AI - Mixture of Agents 2024	layers, agents_per_layer, temperature
process-reward-service	verifySteps	OpenAI Process Reward Models 2023	(none)
conformal-prediction-service	computeBounds	Conformal Prediction for LLMs 2024	alpha, calibration_samples
hitl-review-service	queueForReview	Human-AI Collaboration Patterns	review_type, urgency, auto_approve_threshold
active-learning-service	selectSamples	Active Learning for NLP 2022	strategy, batch_size
pareto-routing-service	selectModel	Pareto-Optimal Model Selection	budget_cents, quality_weight, latency_weight
c3po-cascade-service	selfSupervisedRoute	NeurIPS 2024 - C3PO Self-Supervised	cascade_levels, self_supervised, calibration_samples
automix-service	pomdpRoute	Nov 2025 - AutoMix Self-Improving	pomdp_horizon, exploration_rate, self_verification
cato-neural-decision-service	executeDecision	Internal - Neural decision with affect	safety_mode, use_affect_mapping, use_predictive_coding

Method Aliases

Alias	Maps To
active-sampling-service	active-learning-service
consistency-uq-service	self-consistency-service
tiered-eval-service	hitl-review-service (with review_type: 'tiered')
metaqa-service	multi-hallucination-service (consistency mode)
factual-grounding-service	multi-hallucination-service (attribution mode)

Configurable Parameters Summary

User-Configurable (Think Tank UI)

Category	Parameters
Quality vs. Cost	quality_weight, cost_threshold, budget_cents
Confidence	confidence_threshold, auto_approve_threshold
Sampling	sample_count, temperature, batch_size
Cascade	model_cascade, cascade_levels, max_escalations
Human Review	review_type, urgency

Cato-Configurable (Autonomous)

Category	Parameters
Safety	safety_mode, flag_threshold
Routing	exploration_rate, pomdp_horizon
Synthesis	layers, agents_per_layer, debate_rounds
Verification	self_verification, self_supervised

Saved as User/Tenant Workflows

Workflows with method + parameter configurations can be saved:

```
interface SavedWorkflow {
  workflowId: string;
  tenantId: string;
  userId?: string; // null = tenant-wide
  name: string;
  methodChain: string[];
  methodConfigs: Record<string, Record<string, unknown>>;
  modelOverrides?: WorkflowOverrides;
  scope: 'user' | 'tenant' | 'system';
}
```

70+ System Workflows

System workflows are pre-configured for common use cases:

Category	Example Workflows
Research	deep-research, literature-review, fact-check
Code	code-review, bug-analysis, refactor-proposal
Writing	content-polish, translation, summarization
Analysis	sentiment-analysis, trend-analysis, competitive-intel
Decision	pros-cons-analysis, risk-assessment, recommendation
Creative	brainstorm, story-expansion, concept-generation

Users/Cato decide which AI models are used for any workflow.

Competitive Analysis Request

Gemini, please compare our implementation against:

1. **LangChain/LangGraph** - Graph-based workflow orchestration
2. **AutoGen** - Microsoft's multi-agent framework
3. **CrewAI** - Multi-agent task orchestration
4. **Semantic Kernel** - Microsoft's AI orchestration
5. **DSPy** - Stanford's programming model for LLMs
6. **Anthropic's MCP** - Model Context Protocol

Key differentiators we believe we have: - Model-agnostic condition evaluation - Multi-stream UEP with synthesis
 - 70+ research-backed methods - User-saveable workflow customization - Built-in HITL checkpoints (CP1-CP5) -
 Governance presets (COWBOY/BALANCED/PARANOID)

2030 Readiness Questions

1. **Multimedia orchestration:** How should we prepare for text + image + audio + video workflows?
 2. **Agent-to-agent communication:** Our A2A protocol is basic. What should we add?
 3. **Self-improvement:** AutoMix uses POMDP. What other self-improving patterns should we implement?
 4. **Conditional complexity:** Are our condition types (expression, AI-interpreted, composite) sufficient?
 5. **Stream handling:** Should we support more than 7 parallel streams (current soft limit)?
 6. **Real-time collaboration:** Multiple humans and AIs editing the same workflow. Best approaches?
-

Summary

RADIANT's workflow system provides: - **Composable methods** based on peer-reviewed research - **Multi-AI streams** synthesized through UEP v2.0 - **Model-agnostic conditions** for flexible routing - **User/Cato customization** with saved workflows - **70+ system workflows** for immediate use

We need Gemini's expert opinion on: 1. Conditional node link handling best practices 2. Multimedia stream orchestration patterns 3. Competitive positioning against LangChain, AutoGen, etc. 4. Priority capabilities for 2030 readiness

Document prepared for Gemini consultation - January 31, 2026